

1. 用筛选法求小于200的所有质数。

```
for n ← 2 to 200 do
  for i ← 2 to  $\sqrt{200}$  do
    if n 整除 i
    then n不是质数
    if i >  $\sqrt{n}$ 
    then n是质数
```

2. 对于两个不全为0的非负整数m, n, 用欧几里得算法求m和n的最大公约数。

```
提示: 算法 Euclid(m, n)
// 计算 m 和 n 最大公约数的欧氏算法
// 输入: 两个不全为0的非负整数 m ≥ n
// 输出: m 和 n 的最大公约数
while n ≠ 0 do {
  r ← m mod n
  m ← n
  n ← r
}
return m
```

3. 对于两个给定的n阶方阵A和B, 计算A和B的乘积。

```
提示: MatrixMultiplication (A[0...n-1][0...n-1], B[0...n-1][0...n-1])
for i ← 0 to n-1 do // 行循环
  for j ← 0 to n-1 do // 列循环
    M[i][j] ← 0.0 // 积矩阵初始化
    for k ← 0 to n-1 do // 用变量 k 表示变化的脚标
      M[i][j] ← M[i][j] + A[i][k] * B[k][j]
return M
```

4. 对于数组A[0...n-1], 用冒泡法实现非降序排序。

```
bubble_sort(A[0..n-1]) /*冒泡排序, N是数组a的长度*/
for i ← 0 to n-2 do
  for j ← 0 to n-2-i do
    If A[j+1] < A[j] swap A[j] and A[j+1]
```

5. 给定一个n个字符组成的串(称为文本), 一个m () 个字符组成的串(称为模式), 从文本中寻找匹配模式的子串。

```
Match (T[0..n-1], P[0..m-1])
for i ← 0 to n-m do
  j ← 0
  While j < m and P[j] = T[i+j] do
    j ← j+1
  If j = m return i
return -1
```

6. 对于数组A[0...n-1], 用直接插入排序算法实现非降序排序。

```
Insert(A[0..n-1])
for i ← 1 to n-1 do
  v ← A[i]
```

```

j←i-1
While j≥0 and A[j]>v do
    A[j+1]←A[j]
    j←j-1
A[j+1]←v

```

7. 输入：一个升序数组A[0...n-1]
 和一个查找键K，用折半查找算法实现如下功能，输出：一个数组元素的下标，该元素等于K；如果没有这样一个元素，则返回-1。

提示：int BinarySearch(int a[],int key,int low,int high)

```

{
    int mid=-1;
    if(low<=high)
    {
        mid=(low+high)/2;
        if(key>a[mid])
            mid=BinarySearch(a,key,mid+1,high);
        else if(key<a[mid])
            mid=BinarySearch(a,key,low,mid-1);
    }
    return mid;
}

```

2. 设 a,b,c 是 3 个塔座。开始时，在塔座 a 上有一叠共 n 个圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为 1,2,...,n,现要求将塔座 a 上的这一叠圆盘移到塔座 b 上，并仍按同样顺序叠置。

提示:

$$Hanoi(n, A, B, C) = \begin{cases} move(A, C), n = 1 \\ \begin{cases} Hanoi(n-1, A, C, B) \\ move(A, C) \end{cases} & n > 1 \\ Hanoi(n-1, B, A, C) \end{cases}$$

```
void hanoi(int n, int a, int b, int c)
{
    if (n > 0)
    {
        hanoi(n-1, a, c, b);
        move(a,b);
```

```
        hanoi(n-1, b, a, c);
    }
}
```

3. 分别利用递归代码和非递归代码计算斐波那契数列；比较效率，分析效率差异可能的产生原因。

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

F(n)

If n<=1 return 1

else return F(n-1)+F(n-2)

5. 用从左至右二进制幂算法计算 a^n 。输入：一个数字 a 和二进制位 $b_1, \dots, b_0$ 的列表 b(n)，这些位来自于一个正整数 n 的二进制展开式，输出： a^n 的值。

提示：LeftRightBinaryExponentiation(a,b(n))

//用从左到右二进制幂算法计算 a^n

//输入：一个数字 a 和二进制位 $b_1, \dots, b_0$ 的列表 b(n)，这些位来自于一个正整数 n 的二进制展开式

//输出： a^n 的值，下面的<-表示赋值过程

product <- a

for i <- (l-1) downto 0 do

 product <- product * product

 if b[i] = 1 product <- product * a

return product

6. 用分布计数法，对来自于有限范围整数的一个数组进行排序。输入：数组 $A[0..n-1]$ ，数组中的整数位于 l 和 u 之间 ($l \leq u$)，输出：A 中元素构成的非降序数组 $S[0..n-1]$ 。

提示：ComparisonCountingSort($A[0..n-1]$)

//用比较计数法对数组排序

//输入：可排序数组 $A[0..n-1]$

//输出：将 A 中元素按照升序排列的数组 $S[0..n-1]$

for $i \leftarrow 0$ to $n-1$ do $\text{Count}[i] \leftarrow 0$

for $i \leftarrow 0$ to $n-2$ do

 for $j \leftarrow i+1$ to $n-1$ do

 if $A[i] < A[j]$

$\text{Count}[j] \leftarrow \text{Count}[j] + 1$

 else

$\text{Count}[i] \leftarrow \text{Count}[i] + 1$

for $i \leftarrow 0$ to $n-1$ do $S[\text{Count}[i]] \leftarrow A[i]$

return S

7. 用 Horspool 算法填充移动表。输入：模式 $P[0..m-1]$ 以及一个可能出现字符的字

母表，输出：以字母表中字符为索引的数组 $\text{Table}[0..\text{size}-1]$ 。

提示：ShiftTable($P[0..m-1]$)

//用 Horspool 算法和 Boyer-Moore 算法填充移动表

//输入：模式 $P[0..m-1]$ 以及一个可能出现字符的字母表

//输出：以字母表中字符为索引的数组 $\text{Table}[0..\text{size}-1]$

for $i \leftarrow 0$ to $\text{size}-1$ do $\text{Table}[i] \leftarrow m$

for $j \leftarrow 0$ to $m-2$ do $\text{Table}[P[j]] \leftarrow m-1-j$

return Table

8. 实现 Horspool 字符串匹配算法。输入：模式 $P[0..m-1]$ 和文本 $T[0..n-1]$ ，输出：第一个匹配子串最左端字符的下标，如果没有匹配子串，则返回-1。

提示：HorspoolMatching($P[0..m-1]$, $T[0..n-1]$)

//实现 Horspool 字符串匹配算法

//输入：模式 $p[0..m-1]$ 和文本 $T[0..n-1]$

//输出：第一个匹配子串最左端字符的下标，如果没有匹配子串，则返回-1

ShiftTable($P[0..m-1]$) //生成移动表

$i \leftarrow m-1$ //模式最右端的位置

while $i \leq n-1$ do

$k \leftarrow 0$

 while $k \leq m-1$ and $P[m-1-k]=T[i-k]$ do

$k \leftarrow k+1$

 if $k=m$

 return $i-m+1$

 else $i \leftarrow i+Table[T[i]]$

return -1

8. 对 0-1 背包问题实现记忆功能方法。输入：一个非负整数 i 表示先考虑的物品数量，一个非负整数 j 表示背包的承重量，输出：前 i 个物品的最优可行子集的价值。

答案：

MFKnapsack(i, j) //带记忆功能的递归算法

// $W[1..n]$, $Value[1..n]$, $V[0..n][0..w]$ 全局变量

$V[0..n][0..w] \leftarrow -1$, $V[0][0] \leftarrow 0$

if ($V[i][j] < 0$) //未计算：计算 $V[i][j]$ ，否则不计算，查表得 $V[i, j]$

 if ($j < W[i]$) $V[i][j] \leftarrow MFKnapsack(i-1, j)$

 else $V[i][j] \leftarrow \max(MFKnapsack(i-1, j),$

$Value[i] + MFKnapsack(i-1, j-W[i]))$

return $V[i][j]$ //直接返回结果 (≥ 0 , 查表) 或计算结果 (< 0)

9. 实现传递闭包的 Warshall 算法。输入：包括 n 个顶点有向图的邻接矩阵 A ，输出：该有向图的传递闭包。

答案：

将矩阵 A 赋值给矩阵 $R(0)$

写出三重循环

如果 $R[k][i][j]=1$ 则要么 $R[k-1][i][j]=1$,

要么 $R[k-1][i][k]=1$ 而且 $R[k-1][k][j]=1$ 。)

10: 实现计算完全最短路径的 Floyd 算法。输入：不包含长度为负的回路的图的权重矩阵 W ，输出：包含最短路径长度的距离矩阵。

答案：

将矩阵 W 赋值给矩阵 D

写出三重循环

$D[i][j]$ 的值等于 $D[i][j]$ 或 $D[i][k]+D[k][j]$ 的较小值。

1. 给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 C 。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

提示

```
void Knapsack(int n,float M,float v[],float w[],float x[])
{
    Sort(n,v,w);
    int i;
    for (i=1;i<=n;i++) x[i]=0;
    float c=M;
    for (i=1;i<=n;i++) {
        if (w[i]>c) break;
        x[i]=1;
        c-=w[i];
    }
    if (i<=n) x[i]=c/w[i];
}
```

3. 构造最小生成树的 Prim 算法。输入：加权连通图 $G=\langle V, E \rangle$ ，输出： E_T ，即组成 G 的最小生成树的边的集合。

答案：

PrimMST(G)

$V_T \leftarrow \{v_0\}$

$E_T \leftarrow \Phi$

for ($i \leftarrow 1$ *to* $|V|-1$) *do*

在 $V-V_T$ 中选与当前树距离最小的边 $e^*=(v^*,u^*)$

$V_T \leftarrow V_T \cup \{u^*\}$

$E_T \leftarrow E_T \cup \{e^*\}$

return E_T

5. 实现单起点最短路径的 Dijkstra 算法。输入: 具有非负权重加权连通图 $G=\langle V, E \rangle$ 以及它的顶点 s , 输出: 对于 V 中的每个顶点 v 来说, 从 s 到 v 的最短路径的长度 d_v 。

答案:

```
Initialize(Q)
for (each vertex  $v \in V$ )
     $d_v \leftarrow \infty$ 
    Insert(Q,  $v, d_v$ )
 $d_s \leftarrow 0$ , Decrease(Q,  $s, d_s$ )
 $V_T \leftarrow \Phi$ 
for ( $i \leftarrow 0$  to  $|V| - 1$ )
     $u^* \leftarrow \text{DeleteMin}(Q)$ 
     $V_T \leftarrow V_T \cup \{u^*\}$ 
    for (each vertex  $u \in (V - V_T)$  adjacent to  $u^* \in V_T$ )
        if ( $d_{u^*} + w(u^*, u) < d_u$ )
             $d_u \leftarrow d_{u^*} + w(u^*, u)$ 
            Decrease(Q,  $u, d_u$ )
    . . . . .
```

1. 有一批共 n 个集装箱要装上 2 艘载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且 $\sum_{i=1}^n w_i \leq c_1 + c_2$ 。装载问题要求确定是否有一个合理的装载方案可将这个集装箱装上这 2 艘轮船。如果有，找出一种装载方案。

提示：

```
void backtrack (int i)
```

```
{// 搜索第 i 层结点
  if (i > n) // 到达叶结点
    更新最优解 bestx,bestw;return;
  r -= w[i];
  if (cw + w[i] <= c) {// 搜索左子树
    x[i] = 1;
    cw += w[i];
    backtrack(i + 1);
    cw -= w[i];
  }
  if (cw + r > bestw) {
    x[i] = 0; // 搜索右子树
    backtrack(i + 1);
  }
  r += w[i];
}
```

2. 给定 n 种物品和一背包。物品 i 的重量是 w_i ，其价值为 v_i ，背包的容量为 C 。问应选择装入背包的物品，使得装入背包中物品的总价值最大？

提示：

```
template<class Typew, class Typep>
Typep Knap<Typew, Typep>::Bound(int i)
{// 计算上界
  Typew cleft = c - cw; // 剩余容量
  Typep b = cp;
  // 以物品单位重量价值递减序装入物品
  while (i <= n && w[i] <= cleft) {
    cleft -= w[i];
    b += p[i];
    i++;
  }
  // 装满背包
  if (i <= n) b += p[i]/w[i] * cleft;
  return b;
}
```